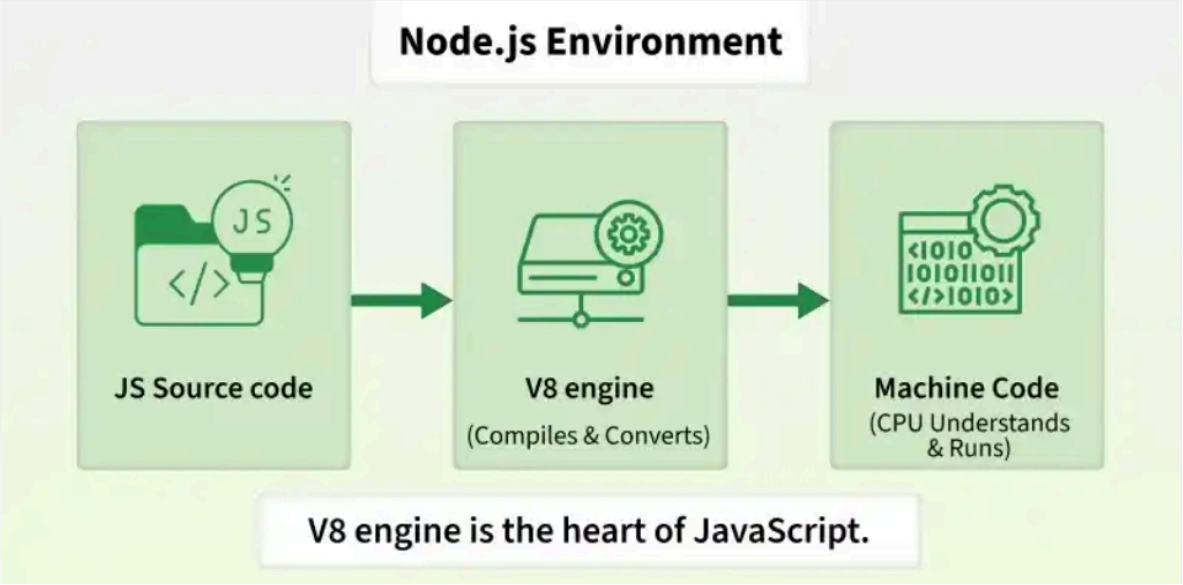
Introduction To Node JS -

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It enables developers to run JavaScript outside the browser to build fast, scalable server-side applications.

- JavaScript was initially a frontend-only language, and Node.js (2009) enabled backend development as well.
- Non-blocking, event-driven architecture for high performance.
- Supports the creation of REST APIs, real-time applications, and microservices.
- Comes with a rich library of modules through npm (Node Package Manager).

*NOTE - Node.js is **not a programming language** and it is **not a framework**. It is a runtime environment for executing JavaScript code without a browser.*



What is the V8 Engine?

The V8 engine is Google's open-source JavaScript engine, used by Chrome and Node.js. Without V8 engine**,** Node.js would not be able to directly execute JavaScript.

It compiles JavaScript to native machine code for fast execution.

- **Origin:** Developed by Google for Chrome in 2008
- **Integration:** Node.js uses V8 to provide a JavaScript runtime on the server
- **Features:** Just-In-Time compilation, ES6+ support

Creation of Node.js -

In **2009**, **Ryan Dahl** introduced Node.js.

His goal was to create a way to use JavaScript for **server-side programming** with an efficient, event-driven architecture.

Node.js used Google's **V8 JavaScript engine** as its JavaScript execution engine.

Installation of Node.js -

Go to official website - https://nodejs.org/en/download/current
Select the latest version and download the installer according to your operating system. Run the installer and keep the default options.

Verify the installation -

After installation, open:
Windows - Command Prompt / PowerShell on VS code
macOS/Linux - Terminal

Run: node -v

You should get something like:
v26.8.1

The exact version will depend on the current LTS release.

First Node.js program -

Create a folder and open it in VS Code.

Create app.js file.

Write: **console.log("Hello from Node.js!");**

Now open the terminal and run:

node app.js

Output:
Hello from Node.js!

That's your first Node program. Here, you will notice that javascript code ran without any browser because it ran with the help of Node.js. It provided the environment(V8 engine) to run the code.

Node provides its own environment and APIs.

For example, Node can interact with the filesystem:

```
const fs = require("fs");
fs.writeFileSync("hello.txt", "Hello Node!");
```

Run: node app.js

A file called: **hello.txt** will be created.

NOTE - JavaScript is now interacting with the computer rather than just the webpage.

Node.js is widely used for:

- Web servers
- REST APIs
- Real-time applications
- Microservices
- Command-line applications
- Backend applications
- Streaming applications

Features of Node.js -

1. JavaScript Runtime -

Node.js allows us to execute JavaScript outside the browser.

```
const name = "Ekta";
console.log(`Hello ${name}`);
```

Run: **node app.js**

2. Built on V8 Engine -

Node.js uses Google's **V8 JavaScript engine** to execute JavaScript.

V8 converts JavaScript into machine-level instructions efficiently.

3. Asynchronous -

Node.js supports **asynchronous operations**.

For example, when Node.js needs to read a file, it doesn't necessarily have to stop the entire application while waiting for the file operation to finish. The program can continue doing other work while the file operation is being completed.

4. Non-Blocking I/O -

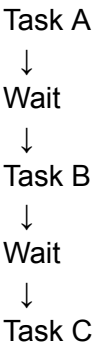
I/O = Input/Output

Examples:

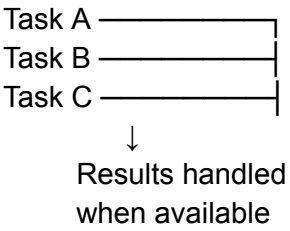
- Reading files
- Writing files
- Database operations
- Network requests

Node.js uses a non-blocking approach for many I/O operations.

Blocking approach



Non-blocking approach



This makes Node.js particularly useful for applications that handle many I/O operations.

5. Event-Driven -

Node.js follows an **event-driven architecture**.

An event can be something like:

- Request received
- File reading completed
- User connected

- Database operation completed

6. Single-Threaded JavaScript Execution -

Node.js executes JavaScript code using a **single main JavaScript thread**. However, this does **not** mean Node.js can only perform one operation at a time. Node.js can handle asynchronous I/O using its event loop and underlying system mechanisms.

7. Fast and Scalable -

Because of its event-driven and non-blocking architecture, Node.js can efficiently handle applications with many concurrent I/O operations.

It is commonly used for:

- Chat applications
- APIs
- Streaming applications
- Real-time applications

8. npm Ecosystem -

npm = Node Package Manager

Node.js has access to a huge ecosystem of reusable packages through **npm (Node Package Manager)**.

It is used to:

- Install packages
- Manage dependencies
- Run scripts
- Manage project configuration

Run on terminal: ***npm -v***

You should also get a version number.

REPL in Node.js -

REPL = Read-Eval-Print Loop

Node.js provides a built-in interactive JavaScript environment called the **Node.js REPL**.
It allows us to execute JavaScript code directly from the terminal without creating a `.js` file.

Starting REPL -

Open the terminal and type:

node

You will see something similar to:

>

Now you can write JavaScript:

> 10 + 20
30

> "Hello".toUpperCase()
'HELLO'

> let x = 100
undefined

> x + 50
150

R — Read
Reads the JavaScript expression entered by the user.

E — Eval
Evaluates/executes the expression.

P — Print
Prints the result.

L — Loop
Waiting for the next command.

REPL is useful for:

- Learning JavaScript
- Testing small pieces of code
- Debugging
- Experimenting with Node.js APIs
- Quickly checking syntax or expressions

Example -

```
> const name = "node"
undefined
> name
'node'
> name.length
4
```

Exit REPL -

You can exit using: **.exit** or press: ***Ctrl + C*** twice.

Global Objects -

Global objects are objects/functions/values that are available throughout a Node.js application without explicitly importing them.

Example 1:

console.log("Hello");

We don't need to import the `console` first.

Example 2:

```
setTimeout(() => {
    console.log("Hello");
}, 2000);
```

`setTimeout()` can be used directly.

Example 3:

console.log(process.version);

The `process` object provides information and control over the current Node.js process. This displays the Node.js version.

Understanding the JavaScript Event Loop

Call Stack • Web APIs • Callback Queue • Microtask Queue • The Event Loop

> 👋 **Before we start**
>
> This is one of the most misunderstood topics in JavaScript — and also one of the most rewarding to truly understand. Once the Event Loop "clicks," async code stops feeling like magic and starts feeling like simple, predictable machinery. Take it one section at a time, run the examples yourself, and you'll get there. Let's go!

1. Core Concepts: The Call Stack & the Global Execution Context

1.1 What is the Global Execution Context (GEC)?

The moment a JavaScript file starts running, the JS engine creates a special container called the **Global Execution Context (GEC)**. Think of it as the "main stage" of your program — it is created first, and it is destroyed last. Every other piece of code runs *inside* it, directly or indirectly.

1.2 What is the Call Stack?

The **Call Stack** is a simple data structure that keeps track of *where the program currently is*. It works on a **LIFO** principle — *Last In, First Out*. Whenever a function is called, JavaScript pushes a new frame onto the stack. When that function finishes (returns), its frame is popped off.

> 💡 **Mental Model**
>
> Imagine a stack of plates. The GEC is the table the stack sits on. Every function call adds a plate on top; every function that finishes removes the plate from the top. You can only ever look at (execute) the very top plate.

1.3 Step-by-Step Trace: A Simple Synchronous Example

Let's trace exactly what the JS engine does with this code:

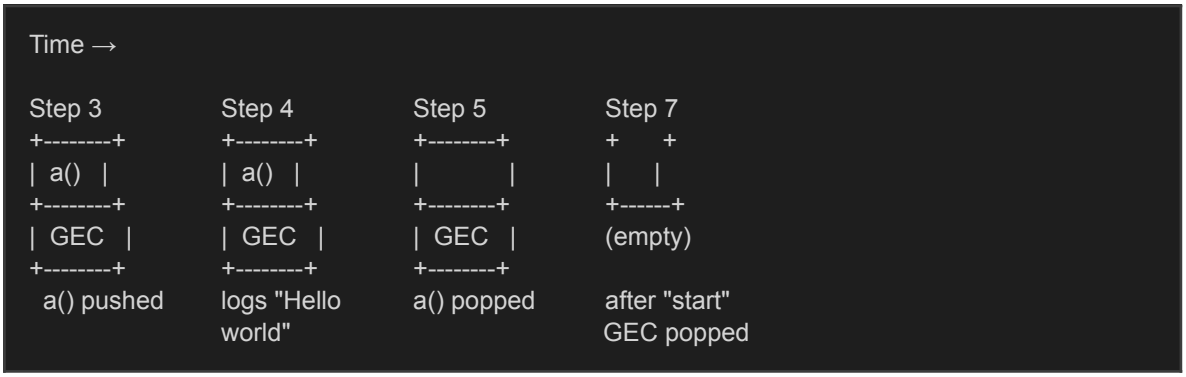
```
function a() {
  console.log("Hello world");
}

a();
console.log("start");
```

Execution trace

1. JS engine starts the file → a Global Execution Context (GEC) is created and pushed onto the Call Stack. Stack: [GEC]
2. Function a() is declared (hoisted) but not run yet — nothing happens on the stack for a declaration.
3. Line a() executes → a new Function Execution Context for a() is created and pushed on top of the GEC. Stack: [GEC, a()]
4. Inside a(), console.log("Hello world") runs → "Hello world" is printed immediately.
5. a() has nothing left to do → its execution context is popped off the stack. Stack: [GEC]
6. Back in the GEC, console.log("start") runs → "start" is printed.
7. The file has no more code → the GEC itself is popped off. Stack: [ ] (empty).

Visualizing the stack over time



// Output:

Hello world
start

> ⚠️ **Common Misconception**
>
> “Functions run in the order they are written.” Not quite — functions run in the order they are CALLED and INVOKED on the stack, which is why a() (called first) fully finishes before console.log("start") even though both lines look sequential. Order comes from the stack, not just the source order.

2. Web APIs / Browser APIs — Why JavaScript Needs Help

2.1 JavaScript itself is single-threaded

Here's the key fact that makes the Event Loop necessary: **the JavaScript engine (like V8) only has ONE Call Stack and can only do ONE thing at a time.** It has *no built-in ability* to "wait" for something (like a timer or a network request) without freezing the entire program.

2.2 So who handles waiting?

JavaScript engines don't run in a vacuum — they run **inside an environment**, such as a **browser** (Chrome, Firefox) or **Node.js**. These environments provide extra capabilities that plain JS does not have, called **Web APIs** in the browser (Node has its own equivalent APIs, e.g. libuv-backed timers and I/O). These include:

- **setTimeout / setInterval** — timers that count down in the background
- **fetch / XMLHttpRequest** — network requests handled outside the JS thread
- **DOM Events** — click, scroll, keypress listeners
- **Geolocation, localStorage, requestAnimationFrame**, and more

> 🔑 **Key Idea**
>
> The browser (or Node) essentially says: "Don't worry, JS — I'll handle the waiting for you. You keep running other code, and I'll let you know the moment the timer finishes / the data arrives." This is what makes JavaScript feel asynchronous even though the engine itself is single-threaded.

2.3 Step-by-Step Trace: Introducing setTimeout

```
console.log("start");

setTimeout(() => {
  console.log("Async Task");
}, 2000);

console.log("end");
```

Execution trace

1. GEC is created and pushed onto the Call Stack.
2. console.log("start") runs → "start" printed immediately.
3. setTimeout(...) is called. JS does NOT run the timer itself — it hands the callback function and the 2000ms delay off to the Web API environment (the

browser), and immediately continues to the next line. The setTimeout call itself is popped off the stack right away.

4. console.log("end") runs → "end" printed immediately.
5. The main script finishes → GEC is popped → Call Stack is empty.
6. Meanwhile, the browser's internal timer counts down in the background (this does NOT block the stack).
7. After ~2000ms, the timer completes. The callback () => console.log("Async Task") is moved into the Callback Queue (Task Queue), waiting for its turn.
8. The Event Loop (explained in Section 4) checks: “Is the Call Stack empty?” Yes → it pushes the callback from the queue onto the stack to run it.
9. "Async Task" is finally printed — roughly 2 seconds after the script started, but AFTER "start" and "end".

// Output:

start

end

Async Task (≈ 2 seconds later)

> ⚠️ **Common Misconception**
>
> “setTimeout(fn, 2000) guarantees fn runs at exactly 2000ms.” Not true — 2000ms is the MINIMUM delay before the callback is eligible to move to the queue. If the Call Stack is busy with other code when the timer finishes, the callback has to wait its turn. The delay is a floor, not a promise.

3. Callback Queue vs. Microtask Queue

Once a Web API finishes its job (timer ends, data arrives, etc.), it doesn't throw its callback directly onto the Call Stack. Instead, it places the callback into a *waiting line* — and JavaScript actually has **two** such waiting lines, with different priority.

3.1 The Callback Queue (a.k.a. Macrotask / Task Queue)

Holds callbacks from:

- setTimeout / setInterval
- DOM events (click, keydown, etc.)
- setImmediate (Node.js)
- I/O callbacks in Node.js

3.2 The Microtask Queue (Priority Queue)

Holds callbacks from:

- **Promise**.then() / .catch() / .finally()
- **async/await** (an awaited call resumes as a microtask)
- queueMicrotask()
- **MutationObserver** callbacks

> 🏆 **The Golden Rule**
>
> The Microtask Queue always has priority over the Callback (Macrotask) Queue. After every single task finishes running on the Call Stack, the Event Loop must FULLY drain the entire Microtask Queue — including any NEW microtasks added along the way — before it is allowed to pick even one task from the Callback Queue.

3.3 Visual Comparison

Feature	Callback Queue (Macrotask)	Microtask Queue
Examples	setTimeout, setInterval, DOM events	Promise.then/catch/finally, async/await, queueMicrotask
Priority	Lower — runs after microtasks are empty	Higher — always runs first
How many run per Event Loop tick	Only ONE macrotask is taken per loop cycle	ALL microtasks run, until the queue is fully empty
Nickname	“Task Queue”	“Priority Queue”

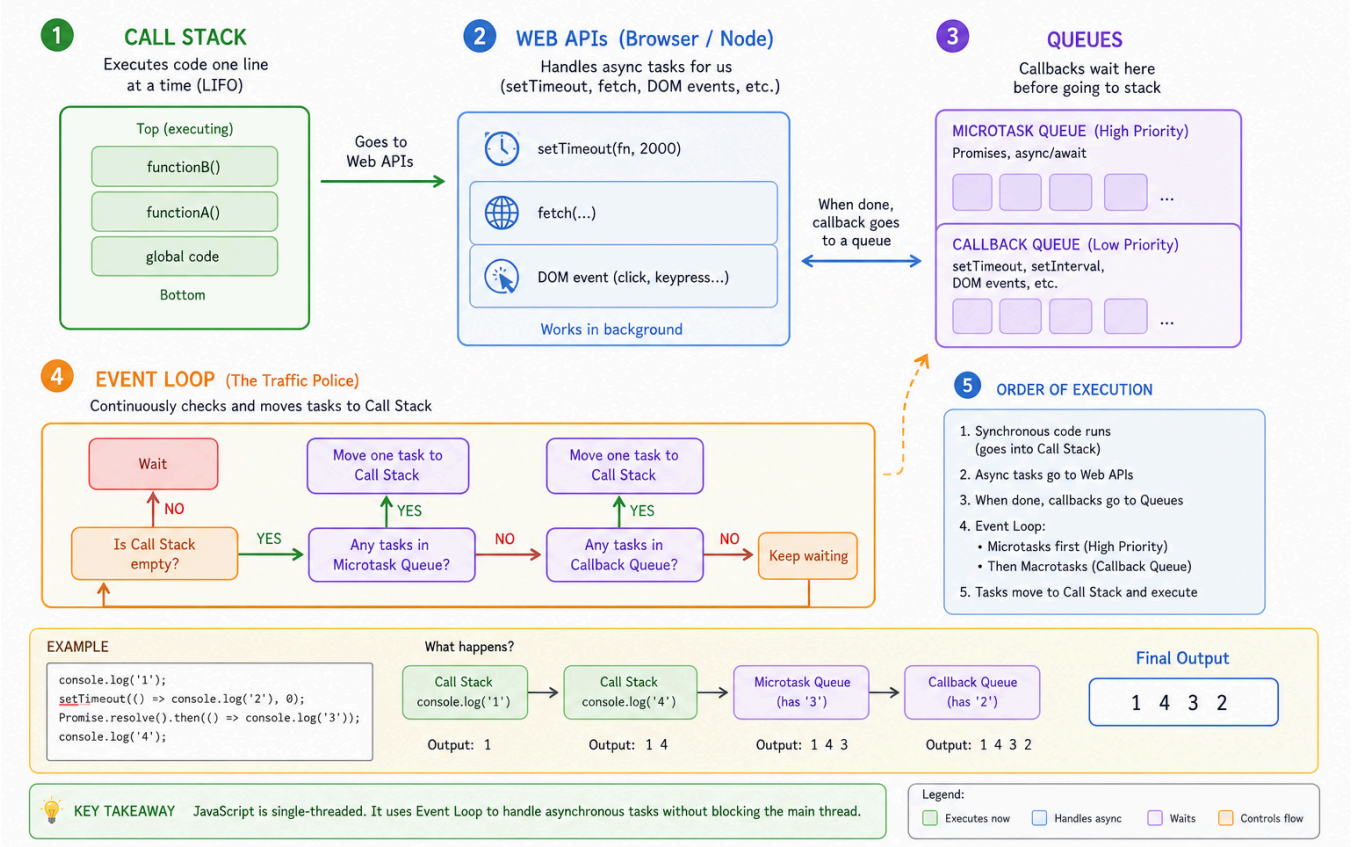
4. The Event Loop — The Bridge That Connects Everything

The **Event Loop** is not a queue itself — it's a constantly-running *watchman / traffic controller*. Its ONE job, repeated forever, is beautifully simple:

> 🔁 **The Event Loop's Job (in plain English)**

> “Is the Call Stack empty? If yes — first, run every single microtask waiting in the Microtask Queue (fully draining it). Then, take ONE task from the Callback Queue and push it onto the Call Stack. Repeat forever.”

4.1 The full picture



4.2 Why the Event Loop matters

Without the Event Loop, JavaScript would have no way to know *when* it's safe to run a queued callback. It is the mechanism that lets a single-threaded language handle thousands of timers, network requests, and UI events without ever blocking the page — as long as the Call Stack is respected: **never run a queued callback while the stack is still busy.**

5. Full Code Walkthroughs

5.1 Walkthrough A — Basic Async Task

```
console.log("1: Sync start");

setTimeout(() => {
  console.log("2: Timeout callback");
}, 0);

Promise.resolve().then(() => {
  console.log("3: Promise callback");
});

console.log("4: Sync end");
```

Trace

1. "1: Sync start" logs immediately (synchronous, runs on the Call Stack).
2. setTimeout hands its callback to the Web API. Even with a 0ms delay, it still must go through the Callback Queue — it cannot run immediately.
3. Promise.resolve().then(...) schedules its callback into the Microtask Queue.
4. "4: Sync end" logs immediately (still synchronous code running).
5. Call Stack is now empty. Event Loop checks the Microtask Queue FIRST → runs it → "3: Promise callback" logs.
6. The Microtask Queue is now empty. Event Loop moves to the Callback Queue → runs the timeout callback → "2: Timeout callback" logs.

// **Output:** 1: Sync start

// **Output:** 4: Sync end

// **Output:** 3: Promise callback

// **Output:** 2: Timeout callback

> ⚠️ **Common Misconception**
>
> “settimeout(fn, 0) runs instantly, right after the current line.” False — even a 0ms setTimeout must go through the Web API and the Callback Queue, so it ALWAYS runs after the synchronous code AND after all microtasks, no matter how short the delay is.

5.2 Walkthrough B — Priority Battle: setTimeout vs. fetch (Promise)

This example shows exactly why the Microtask Queue is called the “Priority Queue” — even when a Promise resolves later than a timer's Web API work theoretically starts, the microtask still cuts in line ahead of the queued macrotask.

```
console.log("A: script start");

setTimeout(() => {
  console.log("B: setTimeout (macrotask)");
}, 0);

fetch("https://example.com/data")
  .then(() => console.log("C: fetch (microtask)"));

Promise.resolve().then(() => console.log("D: promise (microtask)"));

console.log("E: script end");
```

Trace

1. "A: script start" logs — synchronous.
2. setTimeout registers its callback with the Web API and returns immediately (callback destined for the Callback / Macrotask Queue once the timer elapses).
3. fetch() kicks off a network request via the Web API (this takes real time) and immediately returns a pending Promise; .then() registers a microtask to run once that promise resolves.
4. Promise.resolve().then(...) is already resolved, so its callback is queued into the Microtask Queue right away.
5. "E: script end" logs — synchronous code finishes, Call Stack empties.
6. Event Loop drains the Microtask Queue completely before touching the macrotask queue → "D: promise (microtask)" logs first (it was ready immediately).
7. The 0ms setTimeout callback is likely ready in the Callback Queue around this time too, but the Event Loop must fully empty the Microtask Queue on every pass before taking even one macrotask → "B: setTimeout (macrotask)" logs next (assuming the network hasn't responded yet).
8. Once the network response for fetch() actually arrives (this could be milliseconds or longer, and is NOT guaranteed to beat the timer), its microtask joins the Microtask Queue and is drained before any further macrotasks → "C: fetch (microtask)" logs.

// **Output:** A: script start

// **Output:** E: script end

// **Output:** D: promise (microtask)

// **Output:** B: setTimeout (macrotask) <-- typical order, network permitting

// **Output:** C: fetch (microtask) <-- arrives whenever the network responds

> 📌 **Why does D beat B, but C's timing can vary?**
>
> D (Promise.resolve()) is already resolved the instant it's created — so its microtask is ready to run on the very next Event Loop check, ahead of ANY macrotask. B (setTimeout) is a macrotask, so it must wait for the Microtask Queue to be fully empty first. C (fetch) depends on an actual network round-trip — its microtask can only be queued once the response arrives, which is often (but not always) slower than a 0ms timer. The rule that never changes: whenever the microtask queue has something in it, it is drained completely before the next macrotask runs.

Quick-Reference Summary

- **Call Stack** = single-threaded, LIFO, runs synchronous code right now.
- **Web APIs** (browser/Node) = handle waiting (timers, network, DOM) OUTSIDE the JS thread.
- **Callback / Macrotask Queue** = holds setTimeout, DOM event, I/O callbacks. One task per Event Loop cycle.
- **Microtask Queue** = holds Promise/async-await callbacks. ALWAYS fully drained before the next macrotask.
- **Event Loop** = the bridge: “Is the stack empty? Drain microtasks. Then take one macrotask.” Repeat forever.

> 🎉 **You made it!**
>
> If this is your first real pass at the Event Loop, don't worry if it takes a couple of re-reads and a few `console.log` experiments in your own browser console to fully click. Every professional JS developer had this exact same “wait, WHY did that print in that order?!” moment at some point. Trace the examples by hand, predict the output before running the code, and you'll master this faster than you think. Happy coding!

1. Modules in Node.js

A **module** is simply a **separate file containing some code that we can reuse in another file**.

Instead of writing everything in one huge file:

```
app.js
 ├── user code
 ├── database code
 ├── authentication code
 ├── file handling code
 └── utility code
```

we can divide it:

```
project/
│
├── app.js
├── user.js
├── database.js
└── utils.js
```

Each file can act as a **module**.

Why use modules?

- Code becomes easier to understand
- Code can be reused
- Large applications can be divided into smaller parts
- Easier debugging and maintenance

1.1 CommonJS Modules

CommonJS (CJS) is the traditional module system used by Node.js. It uses:

require() to import modules

module.exports to export them. **Example -**

Create: `math.js`

```
function add(a, b) {
    return a + b;
}

function multiply(a, b) {
    return a * b;
}

module.exports = {
    add,
    multiply
};
```

Now create:

app.js

```
const m = require("./math");

console.log(m.add(10, 20));
console.log(m.multiply(5, 4));
```

Output:

30
20

What happened?

math.js
  |
  | module.exports
  ↓
app.js
  |
  | require("./math")
  ↓
Gets the functions

Exporting a single value -

You can also do:

module.exports = add;

Then:

const add = require("./math");

console.log(add(10, 20));

1.2 ES Modules

ES Modules (ESM) is the modern JavaScript module system.

It uses: `export` and `import` instead of `module.exports` and `require()`.

`math.js`

```
export function add(a, b) {
    return a + b;
}

export function multiply(a, b) {
    return a * b;
}
```

`app.js`

```
import { add, multiply } from "./math.js";

console.log(add(10, 20));
console.log(multiply(5, 4));
```

CommonJS vs ES Modules -

CommonJS	ES Modules
Older/traditional Node.js system	Modern JavaScript standard
`require()`	`import`
`module.exports`	`export`
`const fs = require("fs")`	`import fs from "fs"`
Common in older Node projects	Common in modern projects

1.3 Built-in Modules in Node.js

Node.js comes with many modules already. You don't have to install them using npm.

Some important ones are:

fs → File System
path → File and folder paths
http → Create HTTP servers
os → Operating system information
events → Event handling
url → Work with URLs

1.4 File System Module — `fs`

The **File System module (`fs`)** allows Node.js to interact with files and folders.

You can:

- Create files
- Read files
- Write files
- Update files
- Delete files
- Rename files
- Create directories
- Read directories

Import it using CommonJS:

```
const fs = require("fs");
```

1. Reading a File

Suppose we have a file:

`student.txt`

Hello Students!
Welcome to Node.js.

We can read it using:

```
const fs = require("fs");

fs.readFile("student.txt", "utf8", (err, data) => {

    if (err) {
        console.log(err);
        return;
    }

    console.log(data);
});
```

Output:
Hello Students!
Welcome to Node.js.

Understand the parameters:

Here, `fs` is the file system module and `require()` means: **"Give me this module so I can use it in my program."**
fs.readFile("student.txt", "utf8", callback);
student.txt → Which file?
utf8 → How should data be decoded?
`(err,data)` is a callback function which means - Once the file reading is finished, execute this function. Here, `err` stands for **error**. If something goes wrong while reading the file, Node.js puts the error information inside err. `data` contains the **contents of the file** if the file was successfully read.

2. Writing to a File -

```
const fs = require("fs");
fs.writeFile(
    "student.txt",
    "Welcome to Node.js!",
    (err) => {
        if (err) {
            console.log(err);
            return;
        }
        console.log("File written successfully");
```

```
    }
);
```

If `student.txt` doesn't exist, Node.js can create it.If it already exists, `writeFile()` **replaces its contents**.

3. Appending to a File -

If you don't want to replace existing content:

```
const fs = require("fs");

fs.appendFile(
    "student.txt",
    "\nThis is a new line.",
    (err) => {

        if (err) {
            console.log(err);
            return;
        }
        console.log("Data added");
    }
);
```

Now the file becomes:
Welcome to Node.js!
This is a new line.

4. Deleting a File -

```
const fs = require("fs");

fs.unlink("student.txt", (err) => {

    if (err) {
        console.log(err);
        return;
    }
```

```
    console.log("File deleted");
});
```

`unlink()` is commonly used to delete a file.

5. Creating a Folder -

```
const fs = require("fs");
fs.mkdir("students", (err) => {
    if (err) {
        console.log(err);
        return;
    }
    console.log("Folder created");
});
```

6. Reading a Folder -

```
const fs = require("fs");
fs.readdir("students", (err, files) => {

    if (err) {
        console.log(err);
        return;
    }
    console.log(files);
});
```

What if you want the files in the current folder?

```
const fs = require("fs");
fs.readdir(".", (err, files) => {

   if (err) {
      console.log(err);
      return;
   }
   console.log(files);
});
```

Here `"."` means **current folder**.

So you'll get something like:

['app.js', 'student.txt', 'students']

Path Module in Node.js

The **Path module** is a built-in Node.js module that helps us **work with file and folder paths**.

For example:

C:\Users\Ekta\NodeProject\files\student.txt

A path tells us **where a file or folder is located**.

Node.js provides the `path` module so that we can easily create, analyze, and manipulate these paths.

Import the Path Module

With CommonJS:

const path = require("path");

Here:

- `require("path")` → loads Node.js's built-in Path module
- `path` → variable through which we access its functions

Now we can use:

path.join()
path.basename()
path.dirname()
path.extname()
path.parse()

Why do we need the Path Module?

If we want to create this path:

files/student/data.txt

We could write:

const filePath = "files/student/data.txt";

But this can become problematic when working across different operating systems because path separators can differ.

Instead, we can use:

const path = require("path");

const filePath = path.join("files", "student", "data.txt");

console.log(filePath);

Node.js creates the appropriate path for the operating system.

`path.join()` -

This is one of the **most important Path methods**.

const path = require("path");

const filePath = path.join("students", "student", "data.txt");

console.log(filePath);

Output:

students\student\data.txt

`path.join()` automatically handles the correct separator.
So, `path.join()` says:

> **"Node.js, join these folder/file names and create one proper path for me."**

`path.basename()` -

`basename()` gives us the **last part of a path**, usually the filename.

```
const path = require("path");

const filePath = "/students/student/data.txt";

console.log(path.basename(filePath));
```

Output:

data.txt

`path.dirname()` -

`dirname()` gives us the **directory/folder portion** of a path.

```
const path = require("path");

const filePath = "/students/data/student.txt";

console.log(path.dirname(filePath));
```

Output:

/students/data

`path.extname()` -

`extname()` gives us the **file extension**.

```
const path = require("path");

console.log(path.extname("student.txt"));
```

Output:

.txt

Another example:

console.log(path.extname("photo.jpg"));

Output:

.jpg

`path.parse()` -

`parse()` breaks a path into different pieces.

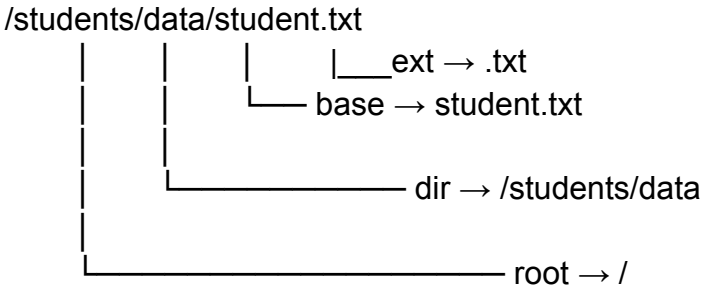
const path = require("path");

const result = path.parse("/students/data/student.txt");

console.log(result);

You'll get an object similar to:

```
{
   root: "/",
   dir: "/students/data",
   base: "student.txt",
   ext: ".txt",
   name: "student"
}
```



So `parse()` gives us **all the important information about a path**.

`path.resolve()` -

It creates an absolute path. **Absolute path** means a complete path that tells you exactly where a file or folder is located, starting from the root of the file system.

For example:

const path = require("path");

const filePath = path.resolve("students", "student.txt");

console.log(filePath);

You might get:

C:\Users\Ekta\NodeProject\students\student.txt

The exact result depends on where your project is located.

Difference:
path.join("students", "student.txt")

creates:

students/student.txt

while:

path.resolve("students", "student.txt")

creates an:

absolute path

`__dirname`

When using **CommonJS**, Node.js provides a special variable called __dirname. It represents the **directory where the current JavaScript file is located**.

Suppose your project is:

```
NodeProject/
│
├── app.js
└── files/
        └── student.txt
```

Inside `app.js`:

console.log(__dirname);

You might get:

C:\Users\Ekta\NodeProject

`__filename`

It gives the **complete path of the current file**.

console.log(__filename);

For example:

C:\Users\Ekta\NodeProject\app.js

```
__dirname  - Folder containing the current file
__filename - Complete path of the current file
```

Method	Purpose	Example
`path.join()`	Joins path pieces	`files/student.txt`
`path.basename()`	Gets filename	`student.txt`
`path.dirname()`	Gets directory	`/files`
`path.extname()`	Gets extension	`.txt`
`path.parse()`	Breaks path into pieces	`{root, dir, base, ext, name}`
`path.resolve()`	Creates absolute path	`C:\Project\files\student.txt`

`fs` works with the actual files and folders, while `path` helps us correctly locate and construct the paths to those files and folders.

What is a Server?

If we just look at the word itself, server is the one that serves something. The system, that is making the request for something, is called the client Server accepts the request from the client and sends the response based on the request.

HTTP

It stands for Hyper Text Transfer Protocol . Set of Protocols or rules that are required to communicate between client and server. HTTPS:- It is exactly the same with an added layer of security, it is achieved by SSL(Secure Socket Layer).

NodeJS HTTP Module

The Node.js HTTP module is a core built-in module used to create and manage HTTP servers and handle client–server communication.

- Included using the require() method.
- Supports standard HTTP methods like GET, POST, PUT, and DELETE.
- Simplifies handling of requests, headers, query parameters, and responses.

```
const http = require("http")
```

Features -

Here are some features of the Node.js HTTP module:

- **Server Creation:** Enables creation of HTTP servers.
- **Request Handling:** Listens for and processes incoming HTTP requests.
- **Response Handling:** Sends responses to clients, supporting dynamic content delivery.
- **HTTP Method Support:** Supports methods such as GET, POST, PUT, DELETE, and others.

Creating Servers using HTTP -

Create a server using http.createServer() that listens for requests.

```
const http=require("http");

const server=http.createServer((req,res)=>{

    res.end("Hello world");

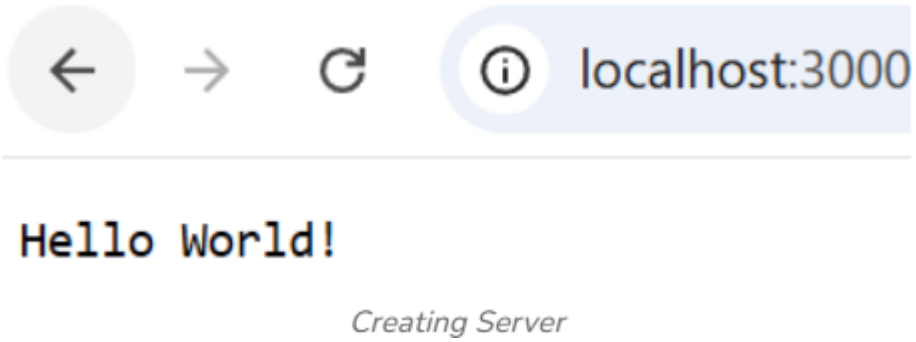
});

server.listen(8080,()=>{

console.log("Running in 8080");

});
```

Output:



Creating Server

Creating HTTP Server with Routing -

We can use Node.js's built-in **HTTP module** to create a server and handle different URLs.

Example -

```
const http = require("http");

const server = http.createServer((req, res) => {
    if (req.url == "/") {

        res.end("Welcome to homePage");

    } else if (req.url == "/about") {

        res.end("Welcome to About Page");

    } else if (req.url == "/cart") {

        res.end("Welcome to Cart page");
    }
});
server.listen(8080, () => {
    console.log("Running in 8080");
});
```

Managing everything with if-else becomes difficult. Express.js makes server and route handling easier.

Nodemon -

Nodemon is a development tool for Node.js that **automatically restarts your application whenever it detects changes to your files**. Normally, if you run: *node app.js* and then change app.js, you have to stop and restart the server manually. With nodemon:

npx nodemon app.js

It watches your files and automatically restarts the server after you save changes.

Installing nodemon -

We install it as a development dependency:

npm install --save-dev nodemon

Note: If you get an error like `npm` command itself is being blocked by **PowerShell's execution policy**. It means npm.ps1 cannot be loaded because running scripts is disabled on this system. Run this command on terminal -

Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

After nodemon is installed, you can run your current program with:
npx nodemon app.js

We use **`npx` to run a package/executable that is installed in our project**. `npx` finds **nodemon inside `node_modules`** and runs it.

`npm` → install/manage packages
`npx` → run packages

Now, you don't have to stop and restart the server manually. Nodemon does it for you. There are two important files which are created automatically after installation of nodemon -

- **`package.json`** → Contains your project's **metadata, dependencies, and scripts**. It tells Node/npm what packages your project needs.
- **`package-lock.json`** → Records the **exact versions of installed packages and their dependencies**, ensuring the same versions are installed on different systems.

In short:
`package.json` = *What my project needs*
`package-lock.json` = *Exactly which versions were installed*

Introduction to Express JS -

Express is used to make it easier to build web servers and backend applications using Node.js.

Without Express, you can use Node's `http` module, but you have to write more code manually.

With Express, you can easily:

- Create servers
- Handle routes/URLs — `app.get()`, `app.post()`, etc.
- Handle HTTP requests and responses
- Build REST APIs
- Use middleware for things like authentication, logging, JSON handling, etc.
- Connect your backend with databases
- Handle errors

Express is a Node.js framework that simplifies the process of creating servers and building backend applications.

Creating a Server using Express.js -

- **Express.js** is a popular framework for building **web applications and APIs** using Node.js.
- It makes it easier to create and manage a server compared to using Node.js's built-in http module.
- When we create an Express.js application, we set up the **foundation for handling incoming requests** and defining how our application **responds to those requests**.

Analogy -

Assume a **server as a waiter in a restaurant**:

- **Client** → Customer
- **Request** → Customer places an order
- **Server** → Waiter
- **Response** → Waiter brings the requested food

In Node.js, we create a server so that it can receive requests and send responses.

What is localhost?

- localhost refers to your own computer.

- When we run a server on our computer, we can access it through localhost.
- For example:

http://localhost:3000

Here:

- **localhost** → Your own computer
- **3000** → Port on which the server is running

Example: If your Express server is running on port **3000**, opening http://localhost:3000 in the browser sends a request to the server running on your computer.

```
const express=require('express')

const app=express()

app.get("/",(req,res)=>{

    res.send("Welocome to home page")

})

app.listen(8080,()=>console.log("Server started in
http:localhost:8080"))
```

- Express is imported using **require('express')**, and an app instance is created with **express()**.
- A route is defined using the **app.get()** method, which responds with a message when the root URL **(/)** is accessed.
- The **app.listen()** method starts the server and listens on port 8080 for incoming requests.

CRUD Application Using Express.js -

1. Creating a CRUD Application -

Till now, we have learned how to create an Express server, create routes, handle requests and responses, and use middleware.

Now, we will combine these concepts and create a simple **CRUD Application using Express.js**.

CRUD stands for:

- **C** → **Create** → Add new data
- **R** → **Read** → Get existing data
- **U** → **Update** → Modify existing data
- **D** → **Delete** → Remove data

For our application, we will manage **student data**.

For example:

Student

id
name
age
course

We will gradually create APIs to:

Create → Add a student
Read → Get students
Update → Update a student
Delete → Delete a student

We will first create a **basic Express application**, then create the **GET request**, and finally create the **POST request**.

Part 1: Create a Basic Express Application

Before creating CRUD APIs, let's first create a basic Express application.

Step 1: Create a Project -

Create a new folder:

```
express-crud
```

Open the folder in VS Code.

Initialize a Node.js project:

```
npm init -y
```

Install Express:

```
npm install express
```

Step 2: Create server.js -

Create a file: **server.js**

```
const express = require("express"); //Import Express framework

const app = express();              //Create Express application

app.listen(8080, () => {
    console.log("Server is running on
http://localhost:8080/");
});
```

Explanation -

1. Import Express -

```
const express = require("express");
```

express is a Node.js framework used to create web servers and APIs easily.

Here, **require("express")** loads the Express package.

We store it inside the **express** variable.

So we can use Express in our application.

2. Create Express Application -

```
const app = express();
```

express() creates an Express application. We store that application inside the **app** variable. We will use an app to create routes and configure our server.

For example:

app.get(...)
app.post(...)
app.use(...)
app.listen(...)

3. Start the Server -

```
app.listen(8080, () => {
   console.log("Server is running on http://localhost:8080/");
});
```

app.listen() starts our Express server. Here, 8080 is the port number. So our application will run at: *http://localhost:8080/*

The function:

```
() => {
   console.log("Server is running...");
}
```

is a **callback function**. It runs when the server successfully starts.

Part 2: Create Our First GET Request -

Now that our basic Express server is ready, let's create a simple GET request.

Add the following code:

```
const express = require("express");

const app = express();

// GET request

app.get("/", (req, res) => {
```

```
    res.send("Welcome to Express Application");

});

app.listen(8080, () => {

    console.log("Server is running on
http://localhost:8080/");

});
```

Explanation of GET Request -

```
app.get("/", (req, res) => {

    res.send("Welcome to Express Application");

});
```

app.get() is used to create a **GET route**.The GET request is generally used when the client wants to **read or retrieve data** from the server. The "**/**" represents the home route. So when the client sends:

GET http://localhost:8080/

this route will execute.

In (req, res), **req** stands for **request**. It contains information coming from the client.

For example:

- URL
- parameters
- query data
- request body
- headers

res stands for **response**. It is used by the server to send a response back to the client. For example:

res.send("Hello"); or res.json(data);

```
res.send("Welcome to Express Application");
```

res.send() sends a response to the client. In this example, the browser will display:

Welcome to Express Application

What Express gives you -

With Node.js `http`	With Express
Manually check URLs	`app.get()`, `app.post()` etc.
Manually handle many routes	Easy routing
More code	Less code

Part 3: Create a Student Data File -

Now let's start building our CRUD application. Create a file: students.json

Add some student data:

```
[
    {
        "id": 1,
        "name": "Aman"
    },
    {
```

```
        "id": 2,
        "name": "Yogesh"
    },
    {
        "id": 3,
        "name": "Rahul"
    }
]
```

Our project structure will now look like:

```
express-crud
 |
 ├── node_modules
 ├── students.json
 ├── package.json
 ├── package-lock.json
 └── server.js
```

Part 4: Import the File System Module -

We need to read and write data inside **`students.json`**. For this, Node.js provides the **File System (`fs`) module**.

Add:

```
const fs=require("fs")
```

So our code becomes:

```
const express = require("express");

const fs = require("fs");

const app = express();
```

Why do we need `fs`?

The `fs` module allows Node.js to work with files.

We can use it to:

- Read files
- Create files
- Write files
- Update files
- Delete files

In our CRUD application, we will use it to store student data in: students.json

Part 5: Enable JSON Data from Client -

Before creating our POST API, we need to understand one important middleware. Add:

`app.use(express.json());`

So:

```
const express = require("express");

const fs = require("fs");

const app = express();

app.use(express.json());
```

What does `express.json()` do?

When a client sends JSON data in the request body, Express needs to understand that JSON data. For example, the client may send:

{

"name": "Aman",

"age": 20

}

The middleware: **app.use(express.json());**

parses the JSON request body and makes it available through: **req.body**

For example:

console.log(req.body);

can give:

{

name: "Aman",

age: 20

}

Without `express.json()`, Express will not automatically parse incoming JSON request bodies.

Part 6: Create GET API to Read Students -

Now let's create our first actual CRUD API. We want:

GET /students

to return all students.

Add:

```
app.get("/students", (req, res) => {

    const data = fs.readFileSync("students.json", "utf-8");

    const students = JSON.parse(data);

    res.send(students);

});
```

Explanation of GET API -

Step 1: Create the route

app.get("/students", (req, res) => {

This creates a GET API:

GET /students

When the client requests:

http://localhost:8080/students

this function will execute.

Step 2: Read the JSON file -

```
const data = fs.readFileSync("students.json", "utf-8");
```

fs.readFileSync() reads the contents of the file. We are reading: students.json. The `"utf-8"` tells Node.js to convert the file data into readable text. The result is stored in data variable. For example: data may contain:

```
[
  {
    "id": 1,
    "name": "Aman"
  },
  {
    "id": 2,
    "name": "Yogesh"
  }
]
```

At this point, `data` is a **string**.

Step 3: Convert JSON string into JavaScript data -

```
const students = JSON.parse(data);
```

JSON.parse() converts JSON-formatted text into a JavaScript value. So:

JSON string

↓

JSON.parse()

↓

JavaScript array/object

Now `students` is a JavaScript array.

Step 4: Send the students -

```
res.send(students);
```

This sends the student data back to the client. The response will look like:

```
[
  {
    "id": 1,
    "name": "Aman"
  },
  {
    "id": 2,
    "name": "Yogesh"
  },
  {
    "id": 3,
    "name": "Rahul"
  }
]
```

Part 7: Create POST API to Add a Student -

We have completed the **Read** operation. Now let's implement the **Create** operation. We want the client to send a new student:

{

"name": "Vikas"

}

and add it to `students.json`.

Create the following POST API:

```
app.post("/students", (req, res) => {


    const data = fs.readFileSync("students.json",
"utf-8");


    const students = JSON.parse(data);


    console.log(students);


    const newStudent = {
        "id": students.length + 1,
        ...req.body
    };


    students.push(newStudent);
```

```
    fs.writeFileSync(
        "students.json",
        JSON.stringify(students)
    );


    res.send("Student Submitted Successfully");
});
```

Explanation of POST API -

Step 1: Create POST Route -

`app.post("/students", (req, res) => {`

This creates a POST API:

POST /students

POST is generally used when we want to **create/add new data**.

Step 2: Read Existing Students -

`const data = fs.readFileSync("students.json", "utf-8");`

We first read the existing student data.

Why?

Because we don't want to overwrite the existing students. We want to add a new student to them.

Step 3: Convert JSON String -

`const students = JSON.parse(data);`

The file data is initially a string. `JSON.parse()` converts it into a JavaScript array. For example:

[

{ id: 1, name: "Aman" },

{ id: 2, name: "Yogesh" }

]

Step 4: Get Data from Client -

req.body

contains the JSON data sent by the client.

For example, the client sends:

{

"name": "Vikas"

}

Then:

req.body

will contain:

{

name: "Vikas"

}

This works because we already added: app.use(express.json());

Step 5: Create New Student -

```
const newStudent = {
    "id": students.length + 1,
    ...req.body
};
```

Suppose we already have 3 students. Then:

students.length is:

3

Therefore:

students.length + 1 is:

4

So the new student becomes:

```
{
    "id": 4,
    "name": "Vikas"
}
```

What does `...req.body` mean?

The `...` is the **spread operator**. Suppose req.body is:

```
{
    "name": "Vikas"
}
```

Then:

```
{
    "id": 4,
    ...req.body
}
```

becomes:

```
{
    "id": 4,
    "name": "Vikas"
}
```

So we are combining: id + client data = new student

Step 6: Add Student to Array -

```
students.push(newStudent);
```

`push()` adds the new student to the existing array. Before:

[

{ id: 1, name: "Aman" },

{ id: 2, name: "Yogesh" },

{ id: 3, name: "Rahul" }

]

After:

[

{ id: 1, name: "Aman" },

{ id: 2, name: "Yogesh" },

{ id: 3, name: "Rahul" },

{ id: 4, name: "Vikas" }

]

Step 7: Save Updated Data -

```
fs.writeFileSync(
    "students.json",
    JSON.stringify(students)
);
```

After adding the new student to the array, we need to save the updated array back into the file.

`fs.writeFileSync()` writes data into a file.

Why did we use `JSON.stringify()`?

Our `students` variable is a JavaScript array:

[

{ id: 1, name: "Aman" },

{ id: 2, name: "Yogesh" }

]

But the file stores JSON text. Therefore, we convert the JavaScript array back into JSON text:

JavaScript array

↓

JSON.stringify()

↓

JSON string

↓

students.json

Step 8: Send Response -

```
res.send("Student Submitted Successfully");
```

After successfully adding the student, we send a response to the client. The client receives:

Student Submitted Successfully

Complete Application So Far -

At this point, our application supports:

CREATE → POST /students

READ → GET /students

Complete code:

```
const express = require("express");    // Import Express framework
const fs = require("fs");              // Import File System module

const app = express();                 // Create Express application

// Middleware
// Used to read JSON data sent by the client
app.use(express.json());

// GET /
// Basic route
app.get("/", (req, res) => {
    res.send("Welcome to Express Application");
});

// GET /students
// Read all students
app.get("/students", (req, res) => {

    // Read students.json file
```

```
    const data = fs.readFileSync("students.json",
"utf-8");


    // Convert JSON string into JavaScript array
    const students = JSON.parse(data);


    // Send students to client
    res.send(students);
});


// POST /students
// Add a new student
app.post("/students", (req, res) => {
    // Read existing students
    const data = fs.readFileSync("students.json",
"utf-8");
    // Convert JSON string into JavaScript array
    const students = JSON.parse(data);


    // Create new student
    const newStudent = {
        id: students.length + 1,
        ...req.body
```

```
    };

    // Add new student to array
    students.push(newStudent);

    // Convert array to JSON and save in file
    fs.writeFileSync(
        "students.json",
        JSON.stringify(students)
    );

    // Send response
    res.send("Student Submitted Successfully");
});

// Start server
app.listen(8080, () => {
    console.log("Server is running on
http://localhost:8080/");
});
```

Part 8: Create GET API to Get a Single Student -

So far, our GET /students route returns all students. But sometimes the client only wants one specific student — for example, the student whose id is 2.

We want:

```
GET /students/:id
```

to return only that one student.

Add the following code (after the GET /students route):

```
app.get("/students/:id", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const id = Number(req.params.id);
  const student = students.find((s) => s.id === id);

  if (!student) {
    return res.send("Student Not Found");
  }

  res.send(student);
});
```

Explanation of GET Single Student API -

Step 1: Create the route with a dynamic parameter

```
app.get("/students/:id", (req, res) => {
```

The :id part is called a route parameter. It is a placeholder for any value the client sends in the URL.

For example, when the client requests:

```
GET http://localhost:8080/students/2
```

Express matches this route, and stores 2 as the value of req.params.id.

Step 2: Read and parse the file

```
const data = fs.readFileSync("students.json", "utf-8");
const students = JSON.parse(data);
```

Just like the GET /students route, we read students.json and convert it into a JavaScript array, so we can search through it.

Step 3: Read the id from the URL

```
const id = Number(req.params.id);
```

req.params.id gives us the id from the URL, but it always comes as a string (for example, "2", not 2). Since the id stored inside students.json is a number, we convert it using Number(), so that the comparison in the next step works correctly.

Step 4: Find the matching student

```
const student = students.find((s) => s.id === id);
```

find() loops through the students array and returns the first element that matches the condition.

Here, the condition is:

```
s.id === id
```

This means: "find the student whose id matches the id we got from the URL."

If a match is found, student holds that student's object. If no student matches, student becomes undefined.

Step 5: Handle the case when the student is not found

```
if (!student) {
  return res.send("Student Not Found");
}
```

If student is undefined, we immediately send a "Student Not Found" message and return, so the code below does not run.

Step 6: Send the student

```
res.send(student);
```

If a matching student was found, we send it back to the client. For example, requesting:

```
GET http://localhost:8080/students/2
```

will return:

```
{
  "id": 2,
  "name": "Yogesh"
}
```

Part 9: Create PUT API to Update a Student -

We have completed Create and Read. Now let's implement the Update operation using a PUT request.

PUT is generally used when we want to update/modify existing data.

We want the client to send updated data, for example:

```
{
  "name": "Vikas Kumar"
}
```

to this URL:

```
PUT /students/4
```

and update the student whose id is 4.

Create the following PUT API:

```
app.put("/students/:id", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const id = Number(req.params.id);
  const index = students.findIndex((s) => s.id === id);

  if (index === -1) {
    return res.send("Student Not Found");
  }

  students[index] = {
    ...students[index],
    ...req.body
  };

  fs.writeFileSync(
    "students.json",
    JSON.stringify(students)
  );

  res.send("Student Updated Successfully");
});
```

Explanation of PUT API -

Step 1: Create PUT Route -

```
app.put("/students/:id", (req, res) => {
```

app.put() is used to create a PUT route. Just like the GET single-student route, :id lets the client tell us which student to update. For example:

```
PUT http://localhost:8080/students/4
```

Step 2: Read Existing Students -

```
const data = fs.readFileSync("students.json", "utf-8");
const students = JSON.parse(data);
```

As always, we first read the file and convert it into a JavaScript array, because we need to work with the existing data before changing it.

Step 3: Find the Student's Position -

```
const id = Number(req.params.id);
const index = students.findIndex((s) => s.id === id);
```

findIndex() is similar to find(), but instead of returning the matching object, it returns the position (index) of that object inside the array.

For example, if the student with id 4 is the 4th element in the array, index will be:

```
3
```

(because array positions start from 0)

If no student matches, findIndex() returns:

```
-1
```

Step 4: Handle the case when the student is not found

```
if (index === -1) {
  return res.send("Student Not Found");
}
```

If index is -1, it means no student with that id exists, so we send a message and stop, instead of trying to update something that does not exist.

Step 5: Update the Student -

```
students[index] = {
  ...students[index],
  ...req.body
};
```

This line replaces the old student object at that position with a new object.

...students[index] first spreads out the existing student's data (so we don't lose fields the client did not send).

...req.body then spreads the new data sent by the client, overwriting any matching fields.

For example, if the existing student is:

```
{ id: 4, name: "Vikas" }
```

and the client sends:

```
{ "name": "Vikas Kumar" }
```

then the updated student becomes:

```
{ id: 4, name: "Vikas Kumar" }
```

Notice the id stays the same, because req.body did not include an id, so it did not overwrite it.

Step 6: Save Updated Data -

```
fs.writeFileSync(
  "students.json",
  JSON.stringify(students)
```

```
);
```

Just like in the POST API, we convert the updated students array back into JSON text using JSON.stringify(), and save it into students.json using fs.writeFileSync().

Step 7: Send Response -

```
res.send("Student Updated Successfully");
```

After successfully updating the student, we let the client know by sending a confirmation message.

Part 10: Create DELETE API to Delete a Student -

Now let's implement the last CRUD operation: Delete.

DELETE is generally used when we want to remove existing data.

We want:

```
DELETE /students/4
```

to remove the student whose id is 4 from students.json.

Create the following DELETE API:

```
app.delete("/students/:id", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const id = Number(req.params.id);
  const index = students.findIndex((s) => s.id === id);

  if (index === -1) {
    return res.send("Student Not Found");
  }

  students.splice(index, 1);

  fs.writeFileSync(
    "students.json",
    JSON.stringify(students)
  );

  res.send("Student Deleted Successfully");
});
```

Explanation of DELETE API -

Step 1: Create DELETE Route -

```
app.delete("/students/:id", (req, res) => {
```

app.delete() is used to create a DELETE route. Once again, :id tells us which student the client wants to delete.

Step 2: Read Existing Students -

```
const data = fs.readFileSync("students.json", "utf-8");
const students = JSON.parse(data);
```

We read the file and convert it into a JavaScript array, same as before.

Step 3: Find the Student's Position -

```
const id = Number(req.params.id);
const index = students.findIndex((s) => s.id === id);
```

We again use findIndex() to locate the position of the student we want to remove.

Step 4: Handle the case when the student is not found

```
if (index === -1) {
  return res.send("Student Not Found");
}
```

If the student does not exist, we stop here and inform the client, instead of trying to delete something that isn't there.

Step 5: Remove the Student -

```
students.splice(index, 1);
```

splice() removes elements from an array.

splice(index, 1) means: "starting at position index, remove 1 element."

For example, if students is:

```
[
  { id: 1, name: "Aman" },
  { id: 2, name: "Yogesh" },
  { id: 3, name: "Rahul" },
  { id: 4, name: "Vikas Kumar" }
]
```

and index is 3 (the position of id 4), then after splice(3, 1), students becomes:

```
[
  { id: 1, name: "Aman" },
  { id: 2, name: "Yogesh" },
  { id: 3, name: "Rahul" }
]
```

Step 6: Save Updated Data -

```
fs.writeFileSync(
  "students.json",
  JSON.stringify(students)
```

```
);
```

We save the array (with the student removed) back into students.json, same as in the POST and PUT APIs.

Step 7: Send Response -

```
res.send("Student Deleted Successfully");
```

Finally, we confirm to the client that the deletion was successful.

Complete CRUD Application -

At this point, our application supports all four CRUD operations:

- CREATE → POST /students
- READ (all) → GET /students
- READ (single) → GET /students/:id
- UPDATE → PUT /students/:id
- DELETE → DELETE /students/:id

Complete code:

```
const express = require("express");   // Import Express
framework
const fs = require("fs");             // Import File System
module
const app = express();                // Create Express
application

// Middleware
// Used to read JSON data sent by the client
app.use(express.json());

// GET /
// Basic route
app.get("/", (req, res) => {
  res.send("Welcome to Express Application");
});

// GET /students
// Read all students
app.get("/students", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);
  res.send(students);
});

// GET /students/:id
// Read a single student
app.get("/students/:id", (req, res) => {
```

```
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const id = Number(req.params.id);
  const student = students.find((s) => s.id === id);

  if (!student) {
    return res.send("Student Not Found");
  }

  res.send(student);
});

// POST /students
// Add a new student
app.post("/students", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const newStudent = {
    id: students.length + 1,
    ...req.body
  };

  students.push(newStudent);

  fs.writeFileSync(
    "students.json",
    JSON.stringify(students)
  );

  res.send("Student Submitted Successfully");
});

// PUT /students/:id
// Update an existing student
app.put("/students/:id", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const id = Number(req.params.id);
  const index = students.findIndex((s) => s.id === id);

  if (index === -1) {
    return res.send("Student Not Found");
  }

  students[index] = {
    ...students[index],
    ...req.body
  };
```

```
  fs.writeFileSync(
    "students.json",
    JSON.stringify(students)
  );

  res.send("Student Updated Successfully");
});

// DELETE /students/:id
// Delete a student
app.delete("/students/:id", (req, res) => {
  const data = fs.readFileSync("students.json", "utf-8");
  const students = JSON.parse(data);

  const id = Number(req.params.id);
  const index = students.findIndex((s) => s.id === id);

  if (index === -1) {
    return res.send("Student Not Found");
  }

  students.splice(index, 1);

  fs.writeFileSync(
    "students.json",
    JSON.stringify(students)
  );

  res.send("Student Deleted Successfully");
});

// Start server
app.listen(8080, () => {
  console.log("Server is running on http://localhost:8080/");
});
```

MIDDLEWARE IN EXPRESS.JS

1. What is Middleware?

Middleware is a function that runs **between the request and the response**. When a client (browser or Postman) sends a request, it does not go directly to the route. It first passes through the middleware, which can check it, change it, or stop it.

Real-case scenario : Think of a security guard at the gate of a college. Before a person enters the campus (the route), the guard checks the ID card (the middleware). If the ID is valid, the person is allowed inside. If not, the person is stopped at the gate and never reaches the campus. In this example, the security guard represents the middleware.

2. Why Do We Need Middleware?

Suppose we have routes like /profile, /orders and /settings, and all of them need the same check (the user must send email and password). Without middleware, we have to copy the same code into every route:

```
app.get("/profile", (req, res) => {
        const { email, password } = req.body
        if (!email || !password) {
        return res.send("Enter Credentials first")
        }
        res.send("Welcome to my profile page")
})

app.get("/orders", (req, res) => {
        const { email, password } = req.body
        if (!email || !password) {
        return res.send("Enter Credentials first")
        }
        res.send("Your orders")
})

// the same checking code is repeated in every route...
```

Problems with this approach:

- The same code is repeated again and again.
- If we want to change the check, we have to change it in every route.
- It is easy to forget the check in one of the routes, which creates a security hole.

With middleware we write the check only once and attach it wherever it is needed. Middleware is commonly used for:

- **Validation:** checking that the data sent by the user is correct.
- **Authentication:** checking whether the user is logged in or allowed to access a page.
- **Logging:** recording which requests are coming to the server.

- **Parsing data:** converting incoming data into a usable format, for example express.json().
- **Error handling:** catching errors and sending a proper response.

3. Syntax of a Middleware

```
const myMiddleware = (req, res, next) => {
        // do some work here (check, change or log the request)
        next()   // pass the request to the next step
}
```

A middleware always receives three things:

Parameter	Meaning
req	Request object. It contains the data sent by the client, such as req.body, req.headers, req.method and req.url.
res	Response object. It is used to send the reply back to the client, such as res.send() and res.status().
next	A function that passes the request to the next middleware or to the final route. If we do not call it, the request stops there.

4. How Middleware Works

A request moves through the middlewares one by one, in the same order in which they are written:

Request → Middleware 1 → Middleware 2 → Route Handler → Response

Every middleware has only **two choices**:

1. **Call next():** the request is fine, so move it forward to the next step.
2. **Send a response** (res.send() etc.): the request is stopped here and the route is never reached.

Important: If a middleware does neither of the two (no next() and no response), the request will hang and the browser will keep loading forever.

5. A Simple Basic Example (Logger Middleware)

This middleware prints the method and URL of every request in the terminal. It does not block anything, it only records the request and moves on.

```
const express = require("express")
const app = express()

// Middleware: prints a message for every request
const logger = (req, res, next) => {
        console.log("Request received:", req.method, req.url)
        next()
}
```

```
app.use(logger)   // apply to the whole application

app.get("/", (req, res) => {
        res.send("Home Page")
})

app.get("/about", (req, res) => {
        res.send("About Page")
})

app.listen(8000, () => {
        console.log("Server is running in http://localhost:8000/")
})
```

Explanation

1. logger is a normal function with req, res and next.
2. Inside it we print the request method (GET, POST, etc.) and the URL using req.method and req.url.
3. next() passes the request forward. Without it, the page would keep loading.
4. app.use(logger) connects the middleware to the whole application, so it runs for every request that comes after this line.

Output

If we open http://localhost:8000/about in the browser:

- **Terminal:** Request received: GET /about
- **Browser:** About Page

Try this: Remove next() from the logger and open the page again. The terminal still prints the message, but the browser keeps loading because the request never reaches the route.

6. Ways to Use a Middleware

Type	How to write	Where it works
Application-level (global)	app.use(middleware)	Every route written after this line.
Route-level	app.get("/profile", middleware, handler)	Only that particular route.
Multiple middlewares	app.get("/profile", mw1, mw2, handler)	Runs from left to right: mw1, then mw2, then handler.
Built-in	app.use(express.json())	Provided by Express itself. Reads JSON data.

Remember: app.use() affects only the routes that are written **below** it. Routes written above it are not affected.

7. Class Example Explained

In class we made a small server with two middlewares: one to validate the data and one to authenticate the user.

```
const express = require("express")
const app = express()

// express can't decide which type of data is coming (json, file, etc.)
// that's why req.body is undefined without this line
app.use(express.json())

const validationMiddleware = (req, res, next) => {
        const { email, password } = req.body
        if (!email || !password) {
        res.send("Enter Credentials first")
        }
        else if (!email.includes("@") || !email.endsWith(".com")) {
        res.send("Enter Valid Email")
        }
        else {
        console.log("User is validated")
        next()   // proceed to the next middleware
        }
}

const authMiddleware = (req, res, next) => {
        const { email, password } = req.body

        const useremail = "user@gmail.com"
        const userpassword = "123"
        if (email == useremail && password == userpassword) {
        next()
        }
        else {
        res.status(400).send("Please enter valid email and password")
        }
}

app.get("/", (req, res) => {
        res.send("Welcome to homepage")
})

// app.use(validationMiddleware)   // connects this with the whole application
// app.use(authMiddleware)

app.post("/profile", validationMiddleware, authMiddleware, (req, res) => {
        res.send("Welcome to my profile page")
})

app.get("/contact", (req, res) => {
        res.send("Welcome to contact page")
})

app.listen(8000, () => {
        console.log("Server is running in http://localhost:8000/")
})
```

Explanation

1. **app.use(express.json()):** Express cannot know in which format the data is coming (JSON, form data, file, etc.), so req.body would be undefined. This built-in middleware tells Express that the data is JSON and fills req.body.
2. **validationMiddleware:** takes email and password from req.body. If any is missing it replies “Enter Credentials first”. If the email has no @ or does not end with .com it replies “Enter Valid Email”. Otherwise it prints “User is validated” and calls next().
3. **authMiddleware:** compares the email and password with the saved user (user@gmail.com and 123). If they match, it calls next(). Otherwise it sends status 400 with an error message.
4. **Public routes:** / and /contact have no middleware, so anyone can open them.
5. **Protected route:** /profile has two middlewares before the handler: validationMiddleware first and then authMiddleware. The profile page is shown only if both call next().
6. **The commented lines:** app.use(validationMiddleware) would apply the middleware to the whole application. We used the route-level way instead, so that only /profile is protected.

Flow of the /profile request

Request → validationMiddleware → authMiddleware → Profile page

If validationMiddleware finds a problem, the request stops there and authMiddleware never runs. The same applies to authMiddleware.

Testing in Postman

Use Body → raw → JSON for the data.

Route	Body	Response
GET /	(none)	Welcome to homepage
GET /contact	(none)	Welcome to contact page
GET /profile	{ }	Enter Credentials first (stopped by validation)
GET /profile	{"email":"user.com","password":"123"}	Enter Valid Email (stopped by validation)
GET /profile	{"email":"user@gmail.com","password":"999"}	Please enter valid email and password (stopped by auth, status 400)
GET /profile	{"email":"user@gmail.com","password":"123"}	Welcome to my profile page

8. Key Points to Remember

- Middleware is a function with (req, res, next) that runs between the request and the response.

- Always do one of two things: call next() or send a response.
- The **order matters**. Middlewares run in the order in which they are written.
- Use app.use() for the whole application and route-level middleware for specific routes.
- express.json() is needed to read JSON data from req.body.
- When you send a response, use return (for example return res.send(...)) so that the code below does not run and next() is not called by mistake.

Common mistakes

- Forgetting next(), so the request hangs.
- Forgetting express.json(), so req.body is undefined.
- Writing the middleware after the route it should protect.
- Sending a response and also calling next(), which causes the error "Cannot set headers after they are sent".

PASSWORD HASHING AND BCRYPT

1. Why Do We Need Password Hashing?

When a user registers on a website, the server saves the email and password in a database. If we save the password exactly as the user typed it (plain text), then anyone who gets access to the database can read every password. This can happen through a hacker attack, a data leak, or even a careless developer or employee.

This is dangerous because many people use the **same password on many websites**. One leak can put their email, social media and bank accounts at risk.

	Plain text storage	**Hashed storage**
Saved password	123456	$2b$10$N9qo8uLOickgx2ZMRZoMye...
If database leaks	Hacker can read the password directly	Hacker only sees a random-looking string
Safe?	No	Yes

Golden rule: Never store passwords in plain text. Always store the hash of the password.

2. What is Hashing?

Hashing is a process that converts any text (like a password) into a fixed-length string of random-looking characters using a mathematical function. The output is called a **hash**.

Properties of a hash:

- **One-way:** we cannot convert the hash back into the original password.
- **Same input gives the same output** (with a simple hash function).

- **A small change gives a completely different output.**
- **Fixed length:** a short or a long password gives a hash of the same length.

Example using the SHA-256 hash function (first 24 characters shown):

Input	Hash
123	a665a45920422f9d417e4867...
1234	03ac674216f3e15c761ee1a5...

Adding just one character changed the whole hash.

Hashing vs Encryption

	Encryption	Hashing
Direction	Two-way (can be decrypted with a key)	One-way (cannot be reversed)
Used for	Data that must be read later, such as messages	Passwords

How login works with hashing

1. **Register:** user types a password → the server hashes it → only the hash is saved.
2. **Login:** user types the password → the server hashes it again → compares it with the saved hash.
3. If both match, the password is correct. The server never needs to know the real password.

3. Problem with Simple Hashing: Why Do We Need Salt?

With a simple hash, the same password always gives the same hash. This creates two problems:

- If two users have the same password, their hashes are also the same, so a hacker can see this.
- Hackers keep ready-made lists (called rainbow tables) of hashes for millions of common passwords. They simply match the leaked hash with their list and find the password.

Salt solves this. A salt is a **random string added to the password before hashing**. Every user gets a different random salt, so even if two users have the same password, their final hashes are different. Ready-made lists become useless.

Password	Random salt	Final hash
123	x9Kp	Hash A

123	Qw2z	Hash B (different)

4. What is bcrypt?

bcrypt is a popular hashing algorithm and Node.js library designed especially for passwords. It is trusted and widely used. Its main features are:

- It **generates the salt automatically** and stores it inside the final hash, so we do not need to manage it.
- It is **slow on purpose**. This does not bother a normal user, but makes it very costly for a hacker to try millions of guesses.
- We can control how slow it is by using **salt rounds**.

Installation:

```
npm install bcrypt
```

Import: const bcrypt = require("bcrypt")

5. Salt Rounds

In bcrypt.hash(password, 10), the number 10 is called the **salt rounds** (also called the cost factor). It decides how much work bcrypt does to create the hash.

- A **higher** number means a stronger and more complex hash, but it also takes more time.
- Every increase of 1 **doubles** the time taken.
- A **very low** number (like 1) is fast but weak.
- A **very high** number (like 20) is very secure but makes the server slow.
- **10** is the most commonly used value because it gives a good balance between speed and security.

6. Two Main Functions of bcrypt

Function	Returns	Used when
bcrypt.hash(password, saltRounds)	The hashed password	Registering a user
bcrypt.compare(password, hashedPassword)	true or false	Logging in a user

Both functions are **asynchronous** (they return a Promise), because hashing takes some time. That is why we use async and await. This way the server is not blocked while the hash is being created.

7. What Does a bcrypt Hash Look Like?

A sample bcrypt hash:

```
2a10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
```

Part	Meaning
$2a$	Version of the bcrypt algorithm (you may also see $2b$)
10	Salt rounds used
Next 22 characters	The random salt (stored inside the hash)
Last 31 characters	The actual hash of the password

Because the salt is stored inside the hash, bcrypt can verify a password later without any extra data.

8. Class Example Explained

```
const express = require("express")
const app = express()
const bcrypt = require("bcrypt")
app.use(express.json())

app.get("/", (req, res) => {
        res.send("Welcome to homepage")
})

let user = {}   // temporary database

app.post("/register", async (req, res) => {
        const { email, password } = req.body
        user = {
        useremail: email,
        userpassword: await bcrypt.hash(password, 10)
        }
        // bcrypt.hash(originalpw, salt_rounds)
        // salt_rounds => 10 ==> strong, complex hash
        //                 1  ==> weak, less complex hash
        res.send(`${JSON.stringify(user)} registered successfully`)
})

app.post("/login", async (req, res) => {
        const { email, password } = req.body
        if (email !== user.useremail) {
        res.send("Email does not exists")
        }
        const check_user = await bcrypt.compare(password, user.userpassword)
        // bcrypt.compare returns true or false
        if (!check_user) {
        res.send("Password doesn't match")
        } else {
        res.send("User Logged In Successfully")
        }
})

app.listen(8000, () => {
        console.log("Server is running in http://localhost:8000/")
```

```
})
```

Register route (POST /register)

1. The email and password are taken from `req.body`.
2. `await bcrypt.hash(password, 10)` converts the password into a hash. `await` is used because hashing takes time. The route function is marked `async` for this reason.
3. The email and the **hashed** password are saved in the `user` object. The original password is never stored.
4. A success message is sent to the client.

Login route (POST /login)

1. The email and password typed by the user are taken from `req.body`.
2. The email is compared with the saved email.
3. `await bcrypt.compare(password, user.userpassword)` checks the typed password against the saved hash and returns `true` or `false`.
4. If it is `false`, the response is “Password doesn't match”. If it is `true`, the user is logged in.

About the user variable: This is only a temporary database, so the data is lost whenever the server restarts. Real applications store users in a database such as MongoDB or MySQL.

Why do we use compare instead of hashing again and checking with ==?

Because bcrypt adds a new random salt every time, hashing the same password twice gives two different results. So we cannot match them with ==. `bcrypt.compare` reads the salt from the saved hash, applies it to the typed password, and then checks whether the result matches.

Testing in Postman

Route	Body	Response
POST /register	{"email":"user@gmail.com","password":"123"}	Shows the email and the hashed password, then “registered successfully”
POST /login	{"email":"user@gmail.com","password":"123"}	User Logged In Successfully
POST /login	{"email":"user@gmail.com","password":"999"}	Password doesn't match
POST /login	{"email":"wrong@gmail.com","password":"123"}	Email does not exists

9. Improvement: Use return After Sending a Response

In the class code, when the email is wrong, the response “Email does not exists” is sent, but the code does not stop. It continues and runs `bcrypt.compare`, and then tries to send a second

response. This causes the error “Cannot set headers after they are sent” (or a crash if no user is registered yet). Adding `return` stops the function after the first response:

```
app.post("/login", async (req, res) => {
        const { email, password } = req.body

        if (email !== user.useremail) {
        return res.send("Email does not exist")
        }

        const check_user = await bcrypt.compare(password, user.userpassword)

        if (!check_user) {
        return res.send("Password doesn't match")
        }
        res.send("User Logged In Successfully")
})
```

10. Using bcrypt Inside a Middleware

We can combine both topics. Instead of writing the login check inside the route, we can write it as a middleware and protect any route with it:

```
const authMiddleware = async (req, res, next) => {
        const { email, password } = req.body

        if (email !== user.useremail) {
        return res.status(400).send("Invalid email or password")
        }

        const isMatch = await bcrypt.compare(password, user.userpassword)
        if (!isMatch) {
        return res.status(400).send("Invalid email or password")
        }
        next()   // credentials are correct, go to the route
}

app.get("/profile", authMiddleware, (req, res) => {
        res.send("Welcome to my profile page")
})
```

If the email or password is wrong, the middleware stops the request. If both are right, `next()` lets the request reach the profile page.

11. Best Practices

- Never store or log plain-text passwords.
- Do not send the hashed password back to the client in real applications (our class example does it only to show the hash).
- Use a general message like “Invalid email or password”, so that attackers cannot find out which emails are registered.
- Use a real database instead of a variable.